

# 信息对抗技术

## 网络嗅探与 HTTP 明文传输分析

### 实验报告

姓名：张昌源

学号：2313177

实验日期：2026 年 4 月 30 日

目录

|          |                              |           |
|----------|------------------------------|-----------|
| <b>1</b> | <b>实验概述</b>                  | <b>2</b>  |
| 1.1      | 实验目的与要求 . . . . .            | 2         |
| 1.2      | 实验环境 . . . . .               | 2         |
| 1.3      | 简要原理 . . . . .               | 2         |
| <b>2</b> | <b>Sniffer Pro 尝试过程与失败原因</b> | <b>3</b>  |
| 2.1      | 安装与注册过程 . . . . .            | 3         |
| 2.2      | 运行时出现旧组件依赖问题 . . . . .       | 4         |
| 2.3      | 网卡选择与主界面进入失败 . . . . .       | 5         |
| 2.4      | 转用 Wireshark 的原因 . . . . .   | 5         |
| <b>3</b> | <b>Wireshark 实验过程</b>        | <b>6</b>  |
| 3.1      | 选择回环接口并开始捕获 . . . . .        | 6         |
| 3.2      | 搭建本地 HTTP 登录服务 . . . . .     | 6         |
| 3.3      | 访问 HTTP 测试登录页面 . . . . .     | 8         |
| 3.4      | 过滤并定位 HTTP POST 请求 . . . . . | 8         |
| 3.5      | 分析 HTTP 请求头与请求体 . . . . .    | 9         |
| 3.6      | 解析明文用户名和密码字段 . . . . .       | 10        |
| <b>4</b> | <b>实验结果分析与总结</b>             | <b>10</b> |
| 4.1      | 实验结果 . . . . .               | 10        |
| 4.2      | 为什么能直接看到账号密码 . . . . .       | 11        |
| 4.3      | 实验结论与心得体会 . . . . .          | 11        |

# 1 实验概述

## 1.1 实验目的与要求

本实验围绕网络嗅探工具的使用与 HTTP 明文传输风险展开。实验最初尝试使用课程资料中的 Sniffer Pro 4.7.5 完成抓包分析,但由于该软件版本较旧,在当前 Windows XP 虚拟机环境中出现组件依赖、解码引擎加载和授权限制等问题,无法稳定进入捕获主界面。因此,后续改用 Wireshark 完成等价的数据包捕获、过滤和协议解析操作。

本实验的核心要求不是攻击真实网站或获取真实账号,而是在合法、可控的本机环境中构造一个 HTTP 明文登录场景,通过协议分析工具捕获登录表单提交时产生的 HTTP POST 请求,并观察普通 HTTP 传输中表单字段的明文暴露情况。

## 1.2 实验环境

表 1: 实验环境与工具

| 项目        | 内容                                          |
|-----------|---------------------------------------------|
| 初始工具      | Sniffer Pro 4.7.5、Sniffer Pro 4.70.530 汉化补丁 |
| 替代工具      | Wireshark 4.6.5                             |
| 虚拟机环境     | Windows XP, 用于尝试安装和运行 Sniffer Pro           |
| 主机环境      | Windows 10, 用于运行 Wireshark 和本地 HTTP 服务      |
| Python 版本 | Python 3.13.2                               |
| 捕获接口      | Adapter for loopback traffic capture        |
| 测试地址      | http://127.0.0.1:8000                       |
| 测试账号      | testuser                                    |
| 测试密码      | 123456                                      |

## 1.3 简要原理

Wireshark 是常用的网络协议分析工具,可以捕获并交互式浏览计算机网络中的通信流量。HTTP 是一种无状态的应用层协议,普通 HTTP 请求本身不提供传输层加密保护。浏览器提交表单时,如果使用 application/x-www-form-urlencoded 编码方式,表单数据会以类似 key=value&key=value 的形式放在请求体中。该编码方式只是数据组织格式,不是加密机制。因此,在本实验构造的普通 HTTP 场景下,抓包工具能够直接解析出登录表单中的用户名和密码字段。

本实验的关键观察对象是 HTTP POST 请求,其典型结构如下:

POST /login HTTP/1.1

```
Host: 127.0.0.1:8000
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

username=testuser&password=123456
```

## 2 Sniffer Pro 尝试过程与失败原因

### 2.1 安装与注册过程

实验最初按照课程资料尝试使用 Sniffer Pro 4.7.5。安装过程中先填写 User Information 页面中的用户信息和单位信息，随后进入 Sniffer Pro User Registration 页面，填写姓名、单位、地址、国家、电话、邮箱等注册信息，并输入安装说明中提供的序列号。安装程序随后开始复制文件到默认安装目录，路径为：

```
C:\Program Files\NAI\SnifferNT\Program
```

图 1 展示了 Sniffer Pro 安装程序复制文件的过程。

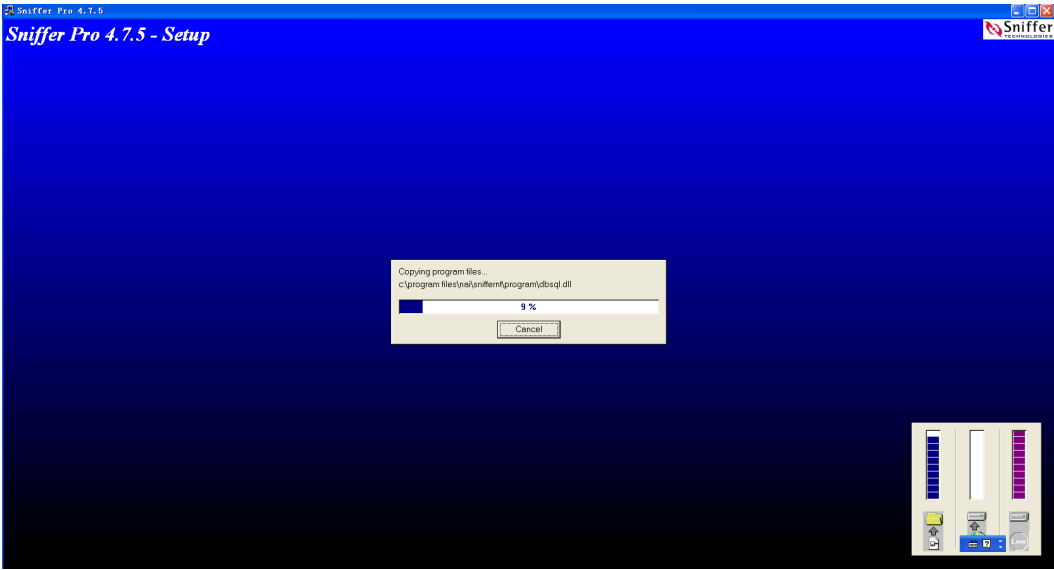


图 1: Sniffer Pro 安装过程中复制程序文件

安装后进入离线注册页面。由于实验环境并不需要联网注册，因此选择离线注册方式，并在注册结果页面中完成安装流程，如图 2 所示。

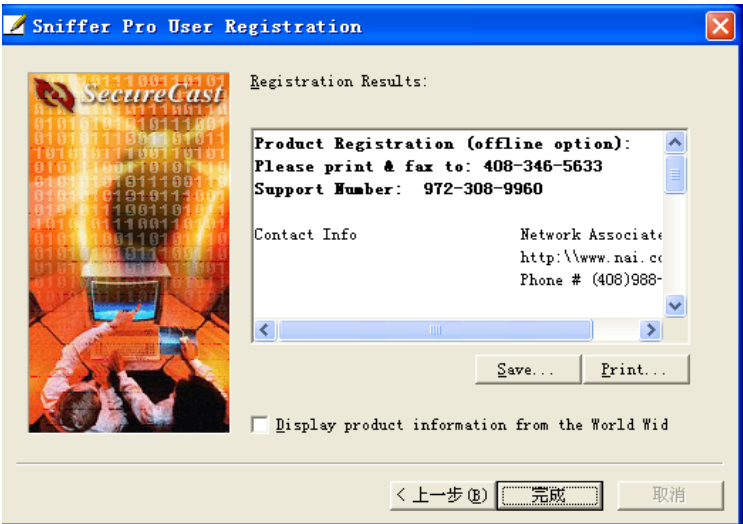


图 2: Sniffer Pro 离线注册结果页面

2.2 运行时出现旧组件依赖问题

安装完成后，首次尝试启动 Sniffer Pro 时，程序提示需要 Microsoft Internet Explorer 5.0 或更高版本，并需要安装 Virtual Machine 组件。该提示说明 Sniffer Pro 4.7.5 作为较旧的软件，对早期 Windows 组件存在依赖，而当前虚拟机环境并未完整提供这些旧组件，如图 3 所示。

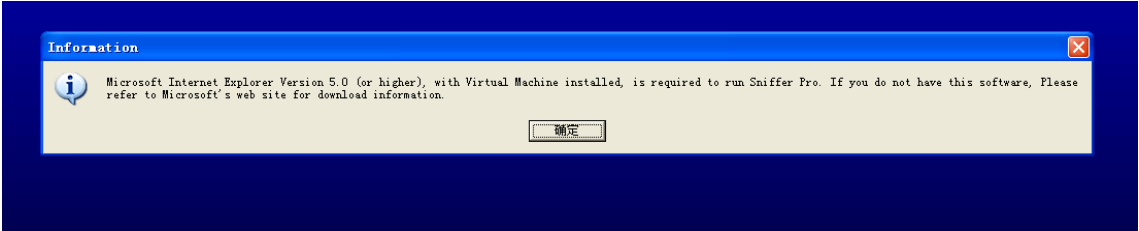


图 3: Sniffer Pro 启动时提示缺少旧版 Virtual Machine 组件

随后尝试运行 Sniffer Portable 版本，希望绕过安装版的旧组件依赖。但 Portable 版本启动时出现“载入 PPI 解码引擎失败”的提示，如图 4 所示。PPI 解码引擎与数据包协议解析功能相关，该错误意味着软件的协议解析模块未能正常加载。



图 4: Sniffer Portable 提示 PPI 解码引擎加载失败

## 2.3 网卡选择与主界面进入失败

在后续尝试中，Sniffer Portable 能够弹出当前设置窗口，并识别到 Windows XP 虚拟机中的虚拟网卡 VMware Accelerated AMD PCNet。按照正常流程，选择网卡后应进入 Sniffer 主界面，然后继续进行捕获和 Decode 分析。但实际操作中，点击“确定”后程序反复回到“当前设置”窗口，无法稳定进入主界面，如图 5 所示。



图 5: Sniffer 当前设置窗口中选择 VMware 虚拟网卡

此外，在误触某些拓扑或分析功能时，程序提示当前 Sniffer Portable 未获得相关功能授权，如图 6 所示。该问题进一步说明当前 Portable 环境功能受限，难以稳定完成课程实验要求中的捕获与解码流程。

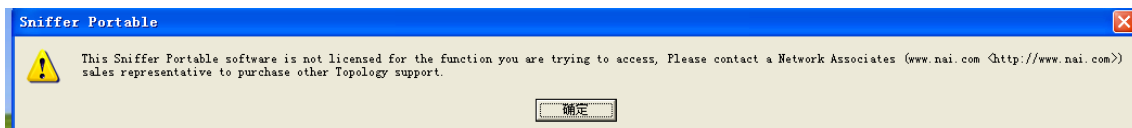


图 6: Sniffer Portable 出现功能授权限制提示

## 2.4 转用 Wireshark 的原因

Sniffer Pro 在本实验环境中主要存在以下问题：

1. 安装版依赖旧版 Microsoft Virtual Machine 组件，当前环境难以直接满足。
2. Portable 版本出现 PPI 解码引擎加载失败，影响协议解析能力。
3. 网卡选择后无法稳定进入主界面，无法继续进行捕获和 Decode 分析。
4. 部分功能存在授权限制，影响正常实验流程。

## 3 Wireshark 实验过程

### 3.1 选择回环接口并开始捕获

由于实验在本机搭建 HTTP 测试服务，浏览器访问地址为 127.0.0.1，通信流量不会经过普通 WLAN 网卡，而是经过本机回环接口。因此，在 Wireshark 主界面中选择 Adapter for loopback traffic capture 作为捕获接口，如图 7 所示。

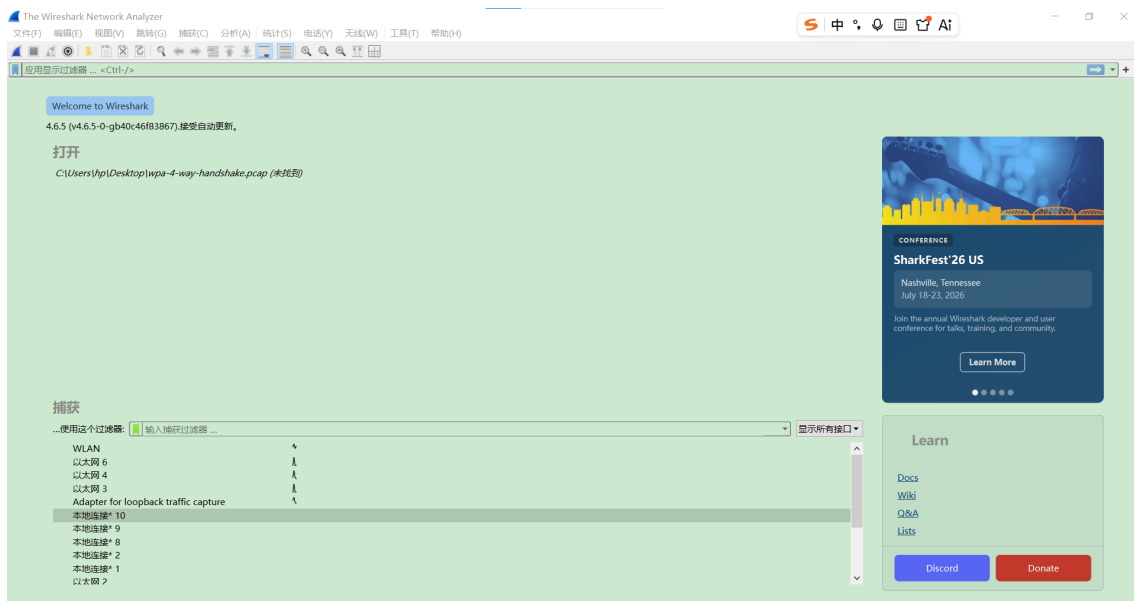


图 7: Wireshark 中选择 Adapter for loopback traffic capture 接口

选择该接口后开始抓包，此时 Wireshark 可以捕获本机浏览器与本机 HTTP 服务之间的通信数据。

### 3.2 搭建本地 HTTP 登录服务

为避免使用真实网站和真实账号，本实验使用 Python 标准库中的 `http.server` 模块搭建一个简单的本地 HTTP 登录页面。程序监听 127.0.0.1:8000，当浏览器访问首页时返回登录表单，当提交表单时通过 `do_POST()` 处理 POST 请求。

测试服务代码如下：

Listing 1: 本地 HTTP 测试登录服务代码

```
from http.server import BaseHTTPRequestHandler, HTTPServer

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        page = """
        <html>
```

```
<head><meta charset="utf-8"><title>HTTP Test Login</title></head>
<body>
<h2>HTTP Test Login</h2>
<form method="POST" action="/login">
    Username: <input name="username"><br>
    Password: <input name="password" type="password"><br>
    <input type="submit" value="Login">
</form>
</body>
</html>
"""
self.send_response(200)
self.send_header("Content-Type", "text/html; charset=utf-8")
self.end_headers()
self.wfile.write(page.encode("utf-8"))

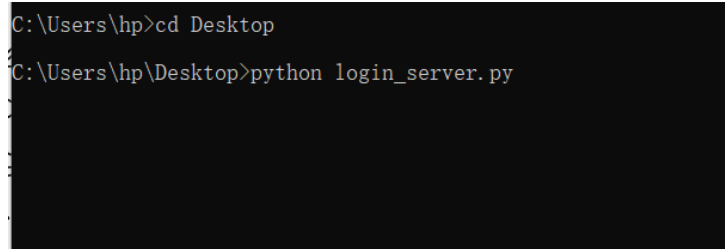
def do_POST(self):
    length = int(self.headers.get("Content-Length", 0))
    body = self.rfile.read(length)
    self.send_response(200)
    self.send_header("Content-Type", "text/html; charset=utf-8")
    self.end_headers()
    self.wfile.write(b"Login data received: " + body)

HTTPServer(("127.0.0.1", 8000), Handler).serve_forever()
```

在命令行中进入桌面目录，并运行该脚本：

```
cd Desktop
python login_server.py
```

运行后命令行窗口保持在程序执行状态，说明本地 HTTP 服务已经启动，如图 8 所示。



```
C:\Users\hp>cd Desktop
C:\Users\hp\Desktop>python login_server.py
```

图 8: 命令行中运行本地 HTTP 测试服务



### 3.3 访问 HTTP 测试登录页面

在浏览器地址栏输入：

```
http://127.0.0.1:8000
```

浏览器成功显示本地 HTTP 测试登录页面，如图 9 所示。随后输入测试账号和密码：

```
Username: testuser
```

```
Password: 123456
```

点击 Login 按钮后，浏览器向 /login 路径发送一个 HTTP POST 请求。

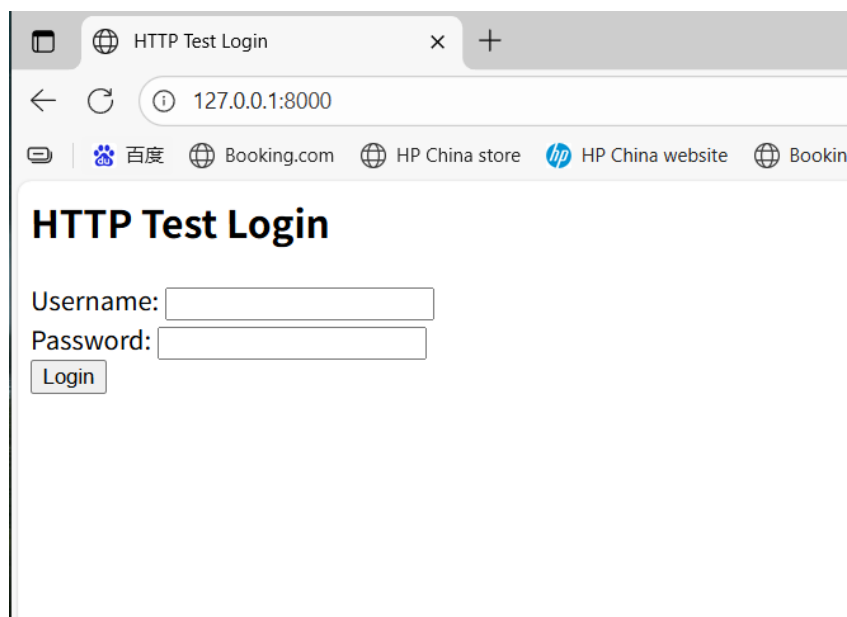


图 9：浏览器访问本地 HTTP 测试登录页面

### 3.4 过滤并定位 HTTP POST 请求

提交登录表单后，回到 Wireshark 停止捕获，并在显示过滤器中输入：

```
http.request.method == "POST"
```

过滤后只保留 HTTP POST 请求，可以看到目标数据包为：

```
POST /login HTTP/1.1
```

该数据包的源地址和目的地址均为 127.0.0.1，说明其属于本机浏览器与本机 HTTP 服务之间的回环通信。过滤结果如图 10 所示。

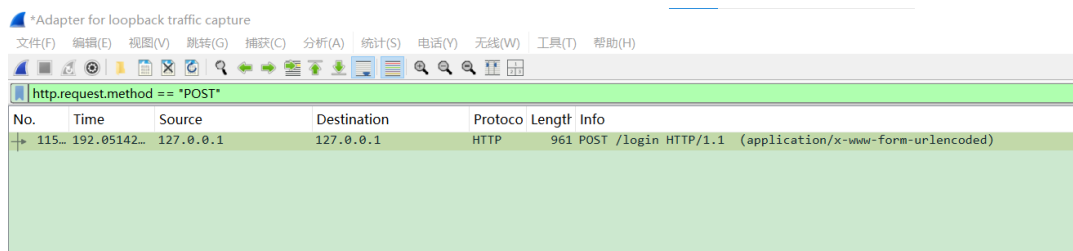


图 10: 使用显示过滤器定位 HTTP POST 请求

### 3.5 分析 HTTP 请求头与请求体

选中 POST /login HTTP/1.1 数据包,在中间的 Packet Details 区域展开 Hypertext Transfer Protocol 字段,可以看到该请求的详细结构,如图 11 所示。

其中,关键字段包括:

- 请求方法: POST
- 请求路径: /login
- 主机地址: 127.0.0.1:8000
- 请求体长度: Content-Length: 33
- 内容类型: Content-Type: application/x-www-form-urlencoded
- 完整请求 URI: http://127.0.0.1:8000/login



图 11: HTTP POST 请求头与请求体信息

该请求的 Content-Type 为 application/x-www-form-urlencoded,说明浏览器以 URL 编码表单格式提交数据。该格式通常将表单组织为:

```
key=value&key=value
```

因此,实验中的表单数据对应为:

```
username=testuser&password=123456
```

### 3.6 解析明文用户名和密码字段

继续展开 HTML Form URL Encoded 字段，Wireshark 直接解析出了表单项目，如图 12 所示：

```
Form item: "username" = "testuser"
Form item: "password" = "123456"
```

这说明用户名和密码没有经过加密处理，而是作为普通 HTTP 请求体中的表单字段被传输。Wireshark 在捕获并解析该 HTTP 数据包后，能够直接显示字段名称和值。



图 12: Wireshark 解析出明文用户名和密码字段

## 4 实验结果分析与总结

### 4.1 实验结果

本实验最终得到如下结果：

1. Sniffer Pro 4.7.5 在当前实验环境中无法稳定进入抓包主界面，因此未能完成正式捕获流程。
2. Wireshark 成功捕获本机回环接口上的 HTTP 登录请求。
3. 使用 `http.request.method == "POST"` 成功定位到 `POST /login HTTP/1.1` 数据包。
4. 在 HTTP 详情中观察到 `Content-Type: application/x-www-form-urlencoded` 和 `Content-Length: 33` 等字段。

5. 展开 HTML Form URL Encoded 后,成功看到 username=testuser 和 password=123456。

## 4.2 为什么能直接看到账号密码

本实验中能够直接看到账号密码,原因在于测试页面使用的是普通 HTTP 协议,请求内容没有经过传输加密。application/x-www-form-urlencoded 只是表单数据的编码方式,它将字段组织为 key=value 并使用 & 分隔多个字段,并不具备加密功能。因此,抓包工具只要捕获到该 HTTP 请求,就可以按照协议格式直接解析请求体内容。

从协议层次看,本次数据包的解析过程如下:

1. 回环接口层:数据在本机 127.0.0.1 上通信。
2. IP 层:源地址与目的地址均为 127.0.0.1。
3. TCP 层:浏览器使用临时端口连接本地服务端口 8000。
4. HTTP 层:浏览器发送 POST /login HTTP/1.1 请求。
5. 表单数据层:请求体中包含 username 和 password 字段。

因此,本实验直观说明了普通 HTTP 明文传输的风险:如果登录凭据在未加密的 HTTP 通信中传输,具备抓包条件的协议分析工具可能直接读取敏感字段。

## 4.3 实验结论与心得体会

本实验完成了从工具安装尝试、问题排查、替代工具选择,到 HTTP 明文登录请求捕获与解析的完整过程。虽然 Sniffer Pro 由于旧版本兼容性问题未能成功完成实验,但这一过程本身也体现了安全实验中环境依赖和工具稳定性的重要性。后续使用 Wireshark 后,实验流程清晰、结果直观,成功观察到明文 HTTP 请求中用户名和密码字段的暴露方式。

通过本实验可以得出以下结论:

1. 网络嗅探工具能够捕获并解析网络接口上的通信数据包。
2. 普通 HTTP POST 请求中的表单数据可能以明文形式出现在请求体中。
3. 表单编码格式并不等同于加密,application/x-www-form-urlencoded 不能保护敏感信息。
4. 在真实系统中,不应使用普通 HTTP 传输用户名、密码等敏感数据。
5. 网络安全实验必须在合法、可控环境中进行,不能抓取真实账号或他人通信内容。

本次实验让我更加直观地理解了“明文传输风险”的含义。过去在理论学习中,明文传输往往只是一个抽象概念;而通过 Wireshark 展开 HTTP 表单字段后,可以直接看到 username 和 password 的值,这使得普通 HTTP 的安全风险变得非常具体。